

操作系统课程设计，2025春

Unix V6++字符设备管理（上 显示屏和显存）

同济大学计算机系



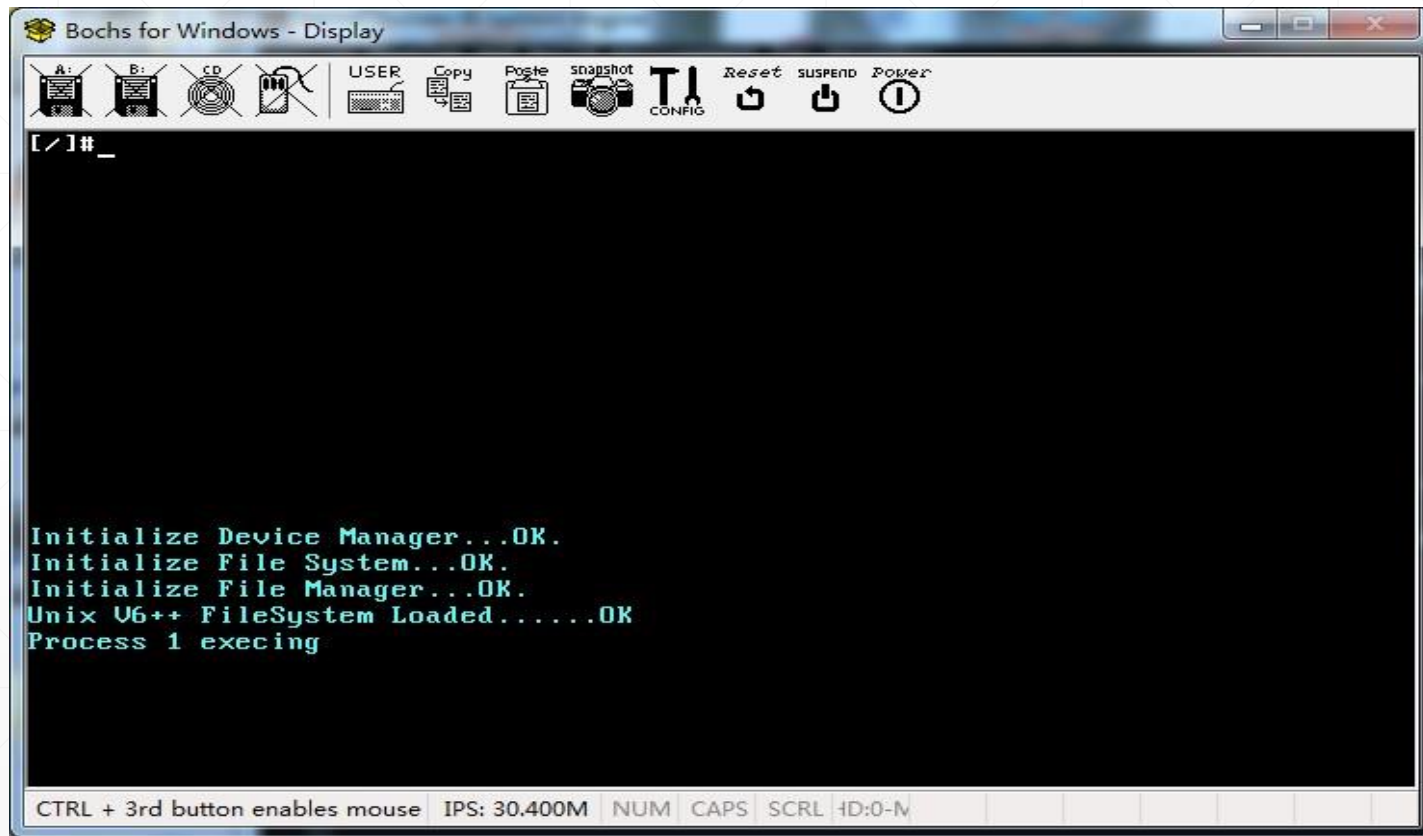
Part 1、显示屏

- 1、使用
- 2、显存
- 3、光标和屏幕输出

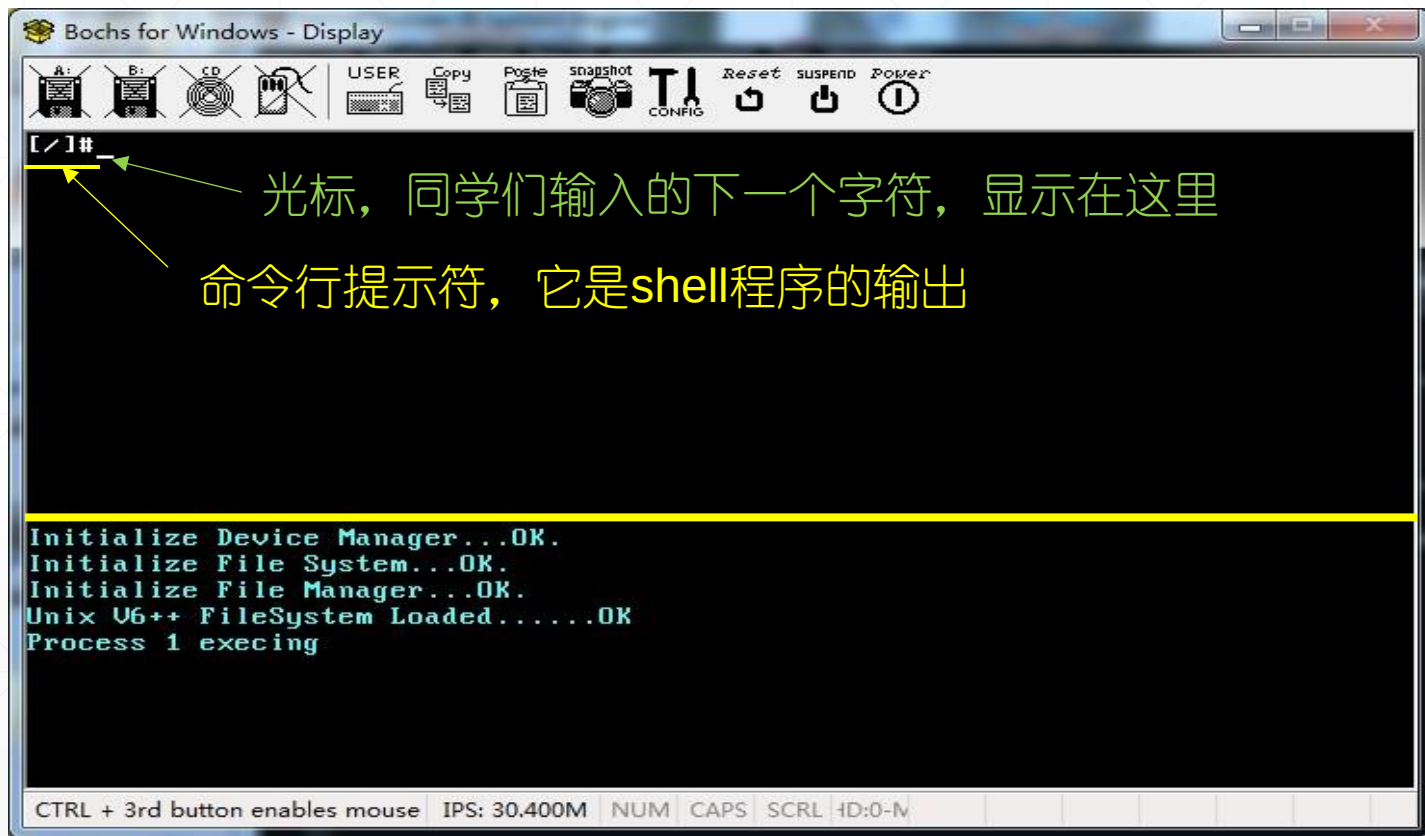
1、Unix V6++ 系统的显示屏

80列, 0 ~ 79

25 行
0 ~ 24



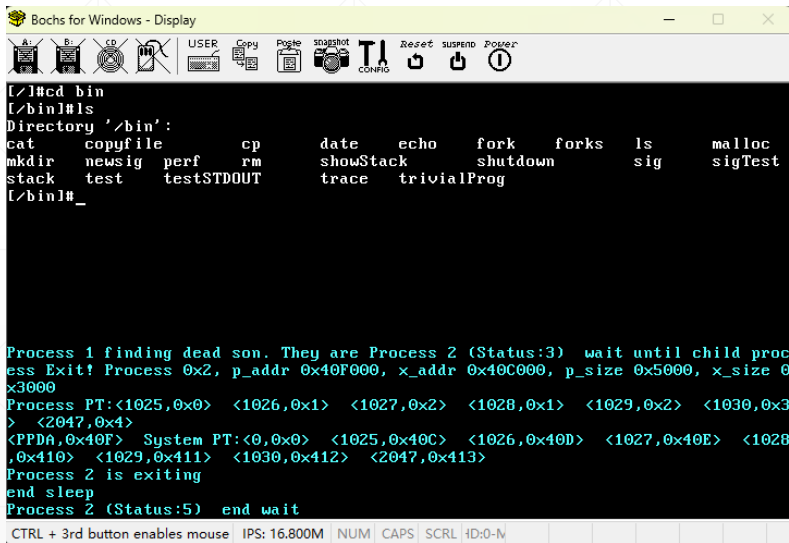
显示屏分2个功能区域



前15 行归正在执行的应用程序使用
显示printf的输出、回显同学们的键盘输入。

后10行供内核输出运行日志（log），
这是重要的调试信息

trace off 和 trace on

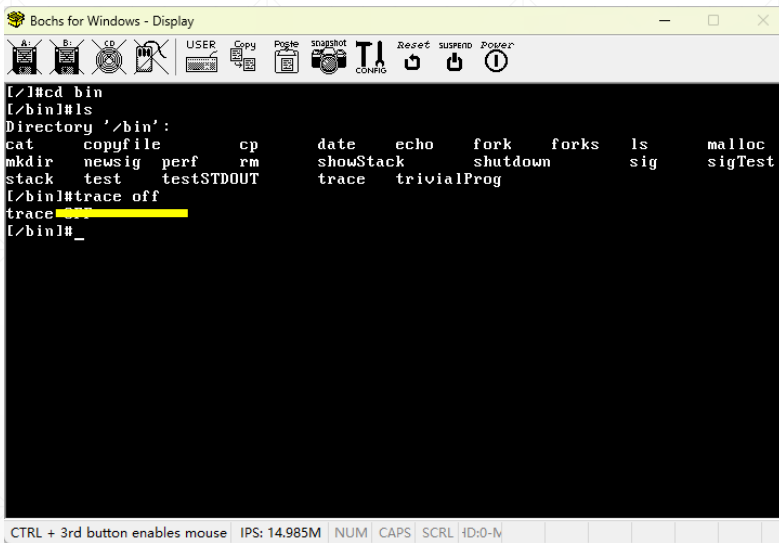


```
Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat    copyfile    cp        date    echo    fork    forks    ls        malloc
mkdir  newsig    perf     rm      showStack  shutdown  sig      sigTest
stack  test     testSTDOUT  trace   trivialProg

[/bin]#_

Process 1 finding dead son. They are Process 2 (Status:3) wait until child proc
ess Exit! Process 0x2, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0
x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3>
> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028
,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
Process 2 is exiting
end sleep
Process 2 (Status:5) end wait
CTRL + 3rd button enables mouse | IPS: 16.800M | NUM | CAPS | SCRL | 4D:0-M
```

(a) 我们一直用的字符界面

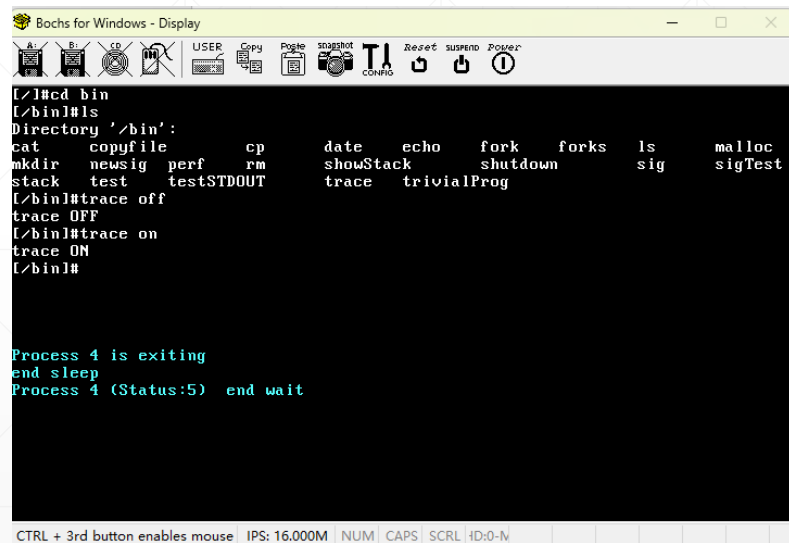


```
Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat    copyfile    cp        date    echo    fork    forks    ls        malloc
mkdir  newsig    perf     rm      showStack  shutdown  sig      sigTest
stack  test     testSTDOUT  trace   trivialProg

[/bin]#trace off
trace OFF
[/bin]#_

CTRL + 3rd button enables mouse | IPS: 14.985M | NUM | CAPS | SCRL | 4D:0-M
```

(b) trace off, 关掉下面



```
Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat    copyfile    cp        date    echo    fork    forks    ls        malloc
mkdir  newsig    perf     rm      showStack  shutdown  sig      sigTest
stack  test     testSTDOUT  trace   trivialProg

[/bin]#trace on
trace ON
[/bin]#_

Process 4 is exiting
end sleep
Process 4 (Status:5) end wait
CTRL + 3rd button enables mouse | IPS: 16.000M | NUM | CAPS | SCRL | 4D:0-M
```

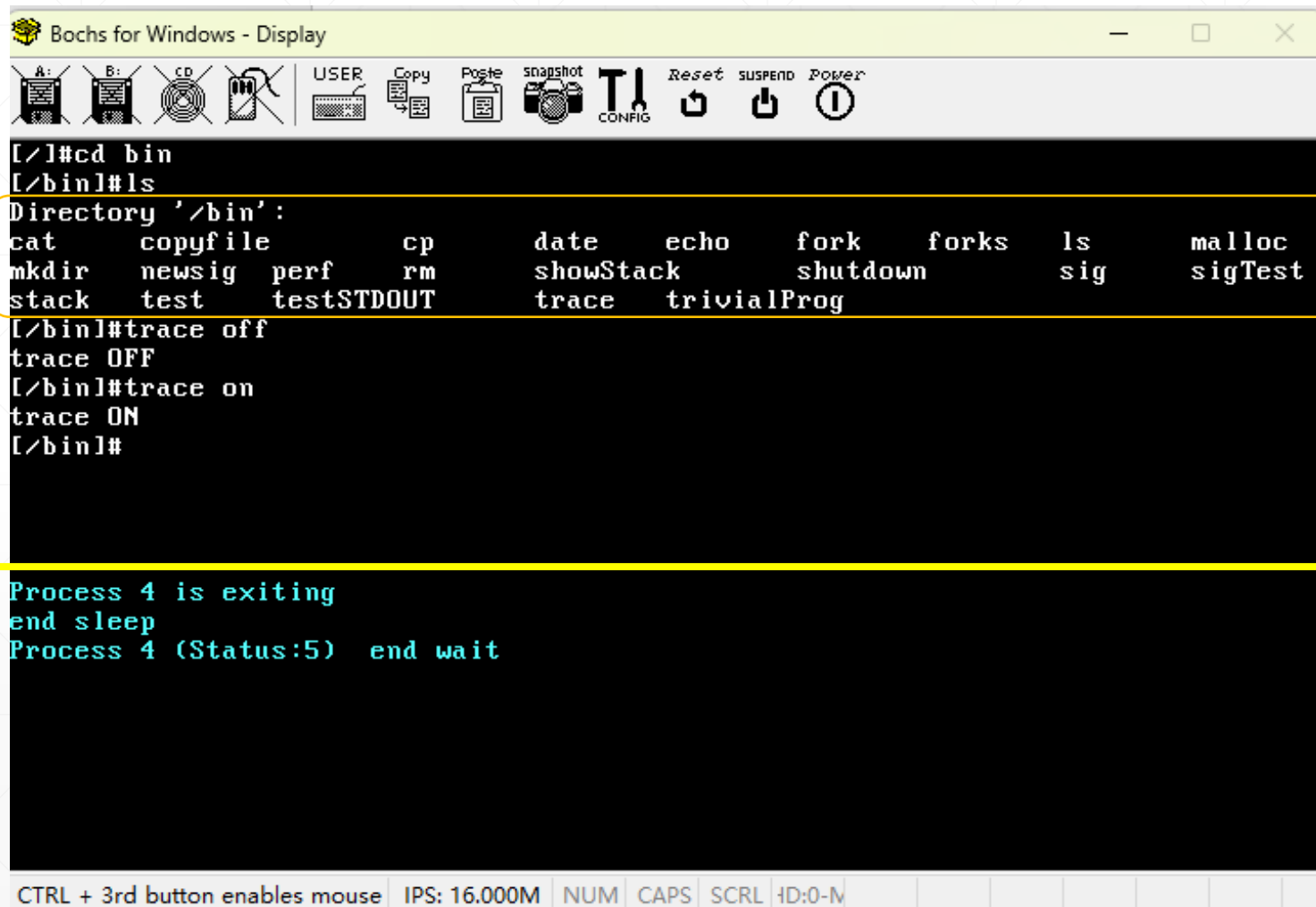
(c) trace on, 复原

上半部是应用程序执行read、write系统调用，读写标准输入输出文件的轨迹。没错，和读写磁盘文件一个style。

命令行提示符 [/]#，
[/bin]# 是shell程序printf
出来的。这里是Shell进
程在写标准输出：
write(1,...,....)。

cd bin, ls, trace off 和
trace on 是用户输入的
命令行，是屏幕在回显
用户输入。

Shell进程执行read系统
调用读标准输入，获得
用户键盘输入的字符串。
进程的0号文件是标准
输出STDIN。这里，
shell进程read(0,...,....)



```
Bochs for Windows - Display
A: B: CD USER Copy Paste snapshot T! CONFIG Reset SUSPEND Power
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat      copyfile      cp      date      echo      fork      forks      ls      malloc
mkdir    newsig    perf      rm      showStack  shutdown  sig      sigTest
stack    test      testSTDOUT trace      trivialProg
[/bin]#trace off
trace OFF
[/bin]#trace on
trace ON
[/bin]#

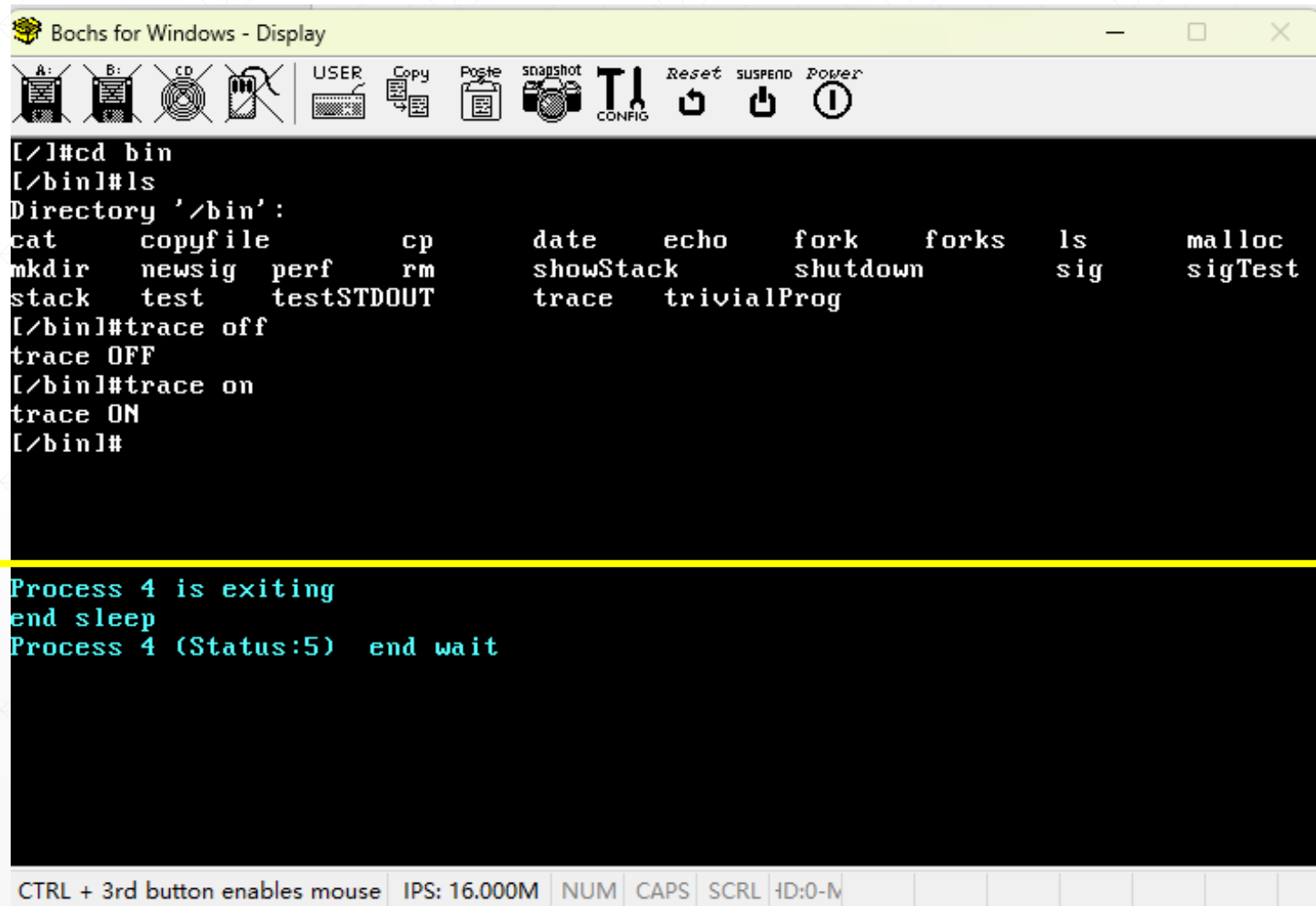
Process 4 is exiting
end sleep
Process 4 (Status:5) end wait

CTRL + 3rd button enables mouse  IPS: 16.000M  NUM  CAPS  SCRL  ID:0-M
```

应用程序ls (ls进程)
的输出，即
printf函数执行系统调
用write(1,...,....)
向屏幕**输出的字符串**

进程的1号文件是标准
输出 STDOUT

下半部，内核写运行日志。



```
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat      copyfile      cp      date      echo      fork      forks      ls      malloc
mkdir    newsig    perf     rm      showStack shutdown  sig      sigTest
stack    test      testSTDOUT
[/bin]#trace off
trace OFF
[/bin]#trace on
trace ON
[/bin]#

Process 4 is exiting
end sleep
Process 4 (Status:5) end wait
```

方法：Diagnose::Write("%s\n", str) 直接写显存。此处，写屏幕不需要借助文件系统。显存是什么，马上会说。

旁白1：观察下，上半部、下半部显示的字体是不一样的。

旁白2：现代的操作系统，比如Linux，内核日志也是往指定磁盘文件写的。

2、显示屏 和 显存

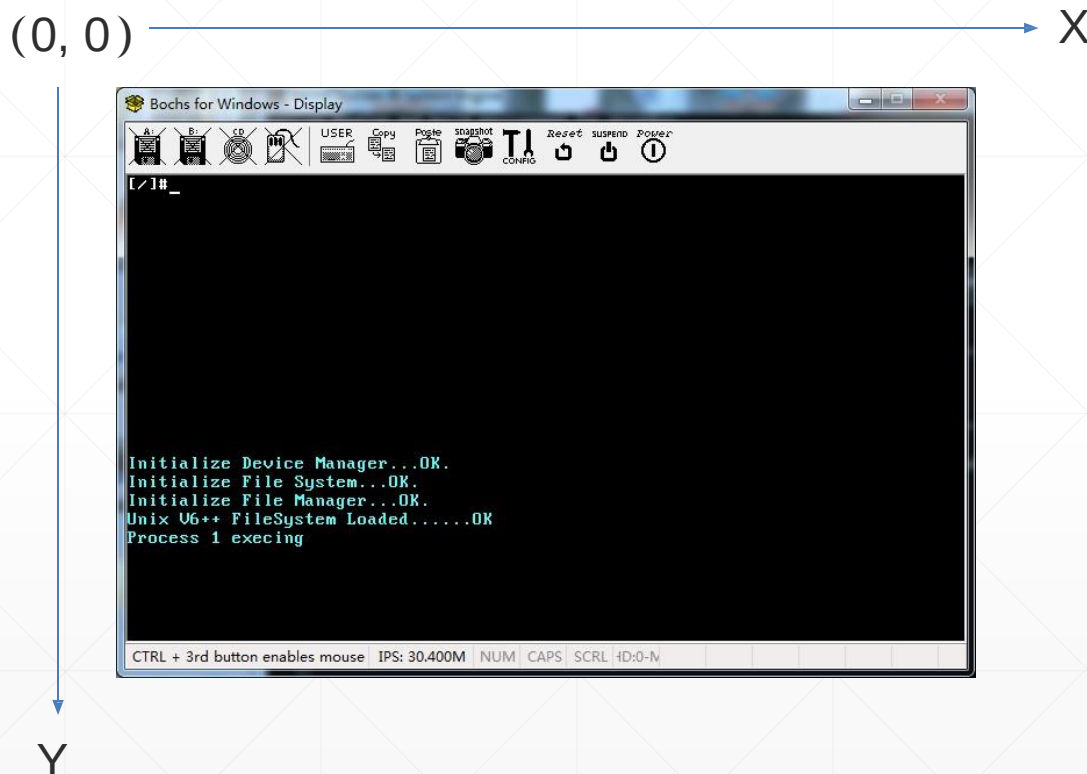
显存是物理内存中的一块固定区域，用来存放屏幕显示的字符。共计 80×25 个单元，每个单元2字节，对应屏幕固定位置，是这个位置显示的字符。第1个字节是字符颜色，第2个字节是字符的ASCII码。

我们用的Bochs虚拟机，使用CRT显示器（阴极射线管）。CRT以固定的频率读显存、刷屏。所以，我们可以认为，内核将字符写入显存，就是在执行屏幕输出操作。

旁白：显存是随机读写的。没错，内核可以把字符写在任意位置，不需要先写前面，再写后面。。。

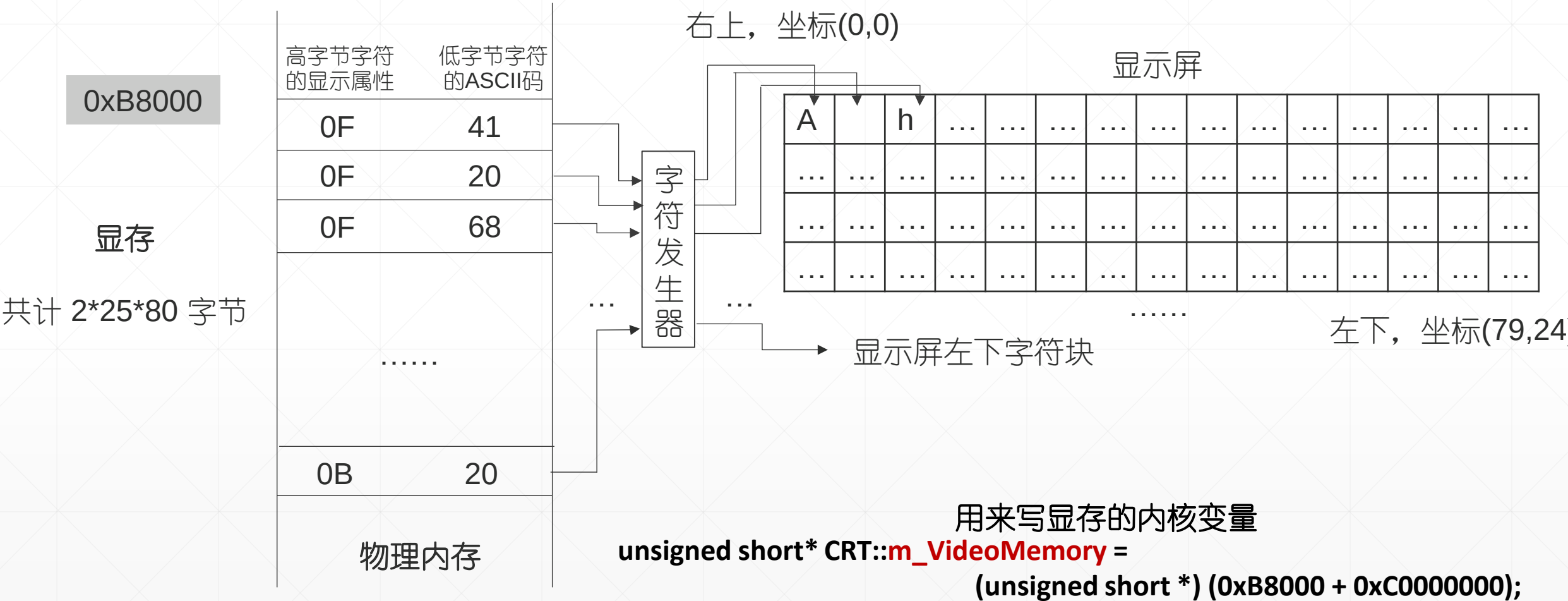


Bochs虚拟机的显示屏

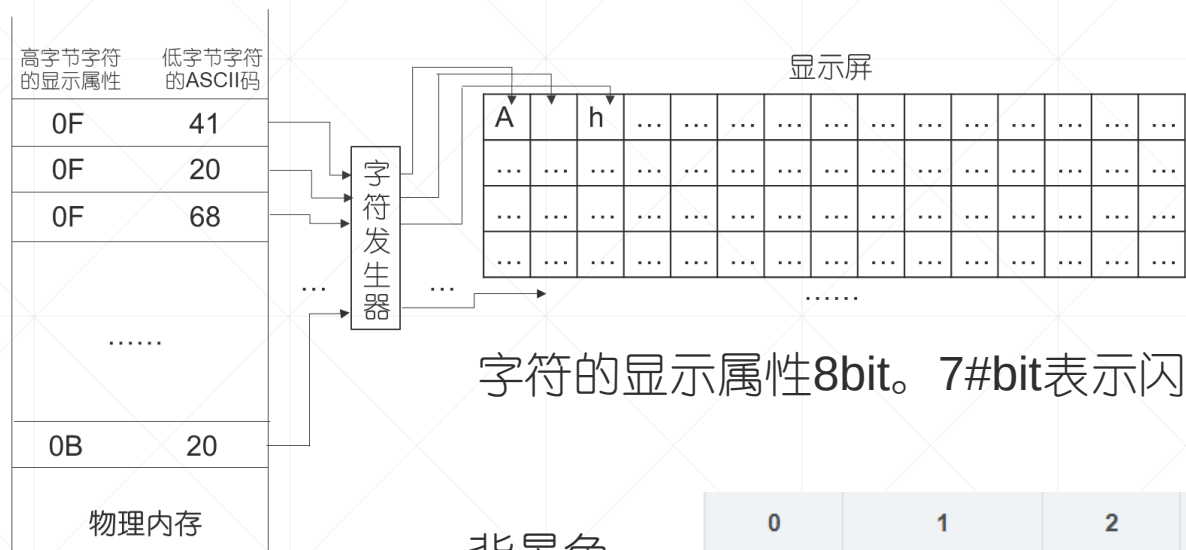


25行，80列。满屏，可以显示 25×80 个字符。

显示屏 和 显存的对应关系



字符的显示属性 和 向屏幕写字符的方法



字符的显示属性8bit。7#bit表示闪烁，4#~6#bit是背景色、字符块的底色，0#~3#bit是前景色、字的颜色。

背景色

0	1	2	3	4	5	6	7
黑	深蓝	绿	青	红	粉红	棕	灰白

前景色

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
黑	深蓝	绿	青	红	粉红	棕	灰白	灰	亮蓝	亮绿	亮青	亮红	亮粉红	黄	亮白

内核向虚地址0xC00B8000写入值0x0F41，显示屏坐标(0,0)，黑底白字显示'A'。0x41，'A'的ASCII码。
 内核向虚地址0xC00B8002写入值0x0F20，显示屏坐标(0,1)，显示一个空格。0x20，空格的ASCII码。
 内核向虚地址0xC00B8004写入值0x0F68，显示屏坐标(0,2)，黑底白字显示'h'。0x68，'h'的ASCII码。



内核实现：CRT类，控制光标、在光标位置写字 crt.h & crt.cpp

crt.h

```
#include "TTY.h"
class CRT
{
    static const unsigned short VIDEO_ADDR_PORT = 0x3d4;
    static const unsigned short VIDEO_DATA_PORT = 0x3d5;
    /* 屏幕大小为80 * 25 */
    static const unsigned int COLUMNS = 80; // 80列
    static unsigned int ROWS;
    static const unsigned short COLOR = 0x0F00; // 黑底白字

    static unsigned short* m_VideoMemory; // 显存首地址
    static unsigned int m_CursorX; // 光标的坐标
    static unsigned int m_CursorY;
    static char* m_Position; // 输出模式：定位tty输出缓存中当前字符，这是CRT对象写入显存的下一个字符。
    static char* m_BeginChar; // 回显模式：tty输出缓存的另一根定位指针，BackSpace不能擦掉这根指针之前的字符。
    .....
};
```

crt.cpp

```
#include "CRT.h"
#include "IOPort.h"

unsigned int CRT::ROWS = 15;
unsigned short* CRT::m_VideoMemory =
    (unsigned short*)(0xB8000 + 0xC0000000);
unsigned int CRT::m_CursorX = 0; 光标的初始位置
unsigned int CRT::m_CursorY = 0;
char* CRT::m_Position = 0;
char* CRT::m_BeginChar = 0;
.....
```

移动光标

- 光标，对应下一个显示字符的坐标 (col,row)。
- 移动光标的方法
 - 计算光标位置:
 $\text{cursorPosition} = \text{row} * 80 + \text{col}$
 - 将cursorPosition的高8位写入CRT 14#寄存器，低8位写入15#寄存器

```
void CRT::MoveCursor(unsigned int col, unsigned int row)
{
    if ( (col < 0 || col >= CRT::COLUMNS) || (row < 0 || row >= CRT::ROWS) )
    {
        return;    // 坐标不对
    }

    /* 计算光标偏移量 */
    unsigned short cursorPosition = row * CRT::COLUMNS + col;

    /* 光标位置的高8位写入14#寄存器 */
    IOPort::OutByte(CRT::VIDEO_ADDR_PORT, 14);
    IOPort::OutByte(CRT::VIDEO_DATA_PORT, cursorPosition >> 8);
    /* 光标位置的低8位写入15#寄存器 */
    IOPort::OutByte(CRT::VIDEO_ADDR_PORT, 15);
    IOPort::OutByte(CRT::VIDEO_DATA_PORT, cursorPosition & 0xFF);
}
```



清屏

- 遍历显存，写80*25个空格

```
void CRT::ClearScreen()
{
    unsigned int i;

    for ( i = 0; i < COLUMNS * ROWS; i++ )
    {
        m_VideoMemory[i] = (unsigned short)' ' | CRT::COLOR; // 0x0F20
    }
}
```

回车

- 修正光标的坐标，至下一行首部。 MoveCursor, 在显示屏上移动光标。行号超过25, 清屏。

```
void CRT::NextLine()
{
    m_CursorX = 0;
    m_CursorY += 1;

    /* 超出最大行数 */
    if ( m_CursorY >= CRT::ROWS )
    {
        m_CursorY = 0;
        ClearScreen();
    }
    MoveCursor(m_CursorX, m_CursorY);
}
```



写字符，ch是其ASCII码

- 更新显存中与光标对应的那两个字节，写入字符和显示属性。完成后，光标右移一位。列号超过80，回车。

```
void CRT::WriteChar(char ch)
{
    m_VideoMemory[m_CursorY * CRT::COLUMNS + m_CursorX] = (unsigned char) ch | CRT::COLOR;
    m_CursorX++;

    if ( m_CursorX >= CRT::COLUMNS )
        NextLine();
    else
        MoveCursor(m_CursorX, m_CursorY);
}
```



显示退格符BackSpace

```
void CRT::BackSpace()
{
    m_CursorX--; // 更新光标的坐标，列号减1

    if ( m_CursorX < 0 ) // 需要回到上一行
    {
        m_CursorX = CRT::COLUMNS - 1;
        m_CursorY--;
        if ( m_CursorY < 0 )
        {
            m_CursorY = 0;
        }
    }
    MoveCursor(m_CursorX, m_CursorY); // 移动光标

    /* 在光标所在位置填上空格 */
    m_VideoMemory[m_CursorY * COLUMNS + m_CursorX] = ' ' | CRT::COLOR;
}
```




显示制表符Tab

制表符：光标沿当前行向右移动，至下一个8的整数倍列。列号超过80，回车。

```
void CRT::Tab()
{
    m_CursorX &= 0xFFFFFFFF8;      /* 左对齐到前一个Tab边界 */
    m_CursorX += 8;
    if ( m_CursorX >= CRT::COLUMNS )
        NextLine();
    else
        MoveCursor(m_CursorX, m_CursorY);
}
```



刷新终端的输出缓存

- `void CRT::CRTStart(TTy* pTTy)`, 这个函数需要与tty的输出缓存同步稍后介绍。
- 我们先看内核如何显示日志。`include/Video.h` 和 `kernel/Video.cpp`中的Diagnose对象。



Diagnose对象，管理内核日志显示区

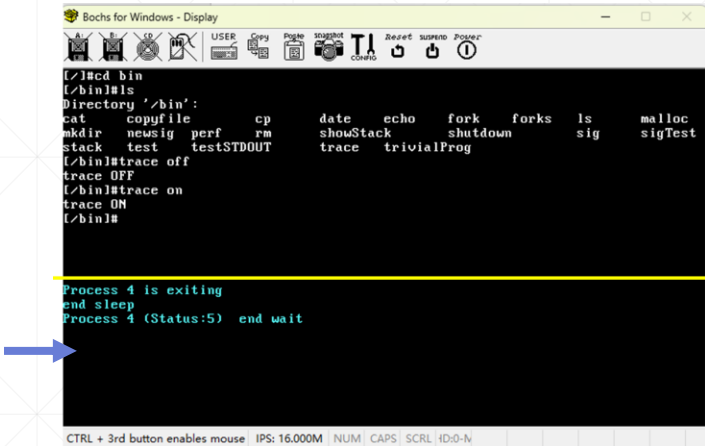
```
// Video.h
#ifndef DIAGNOSE_H
#define DIAGNOSE_H

class Diagnose
{
public:
    static unsigned int ROWS; // 尺寸
    static const unsigned int COLUMNS = 80; // 80列
    static const unsigned short COLOR = 0x0B00; // 黑底CYAN色
    static const unsigned int SCREEN_ROWS = 25; // 全屏，25行

public:
    static unsigned int m_Row; // 内核日志显示区的“光标”
    static unsigned int m_Column;

private:
    static unsigned short* m_VideoMemory; // 显存首地址
    static bool trace_on; // 开启内核日志显示区

};
```





初始化

```
#include "Video.h"
```

```
unsigned short* Diagnose::m_VideoMemory = (unsigned short*)(0xB8000 + 0xC0000000);
```

```
unsigned int Diagnose::m_Row = 10;
```

```
unsigned int Diagnose::m_Column = 0;
```

```
unsigned int Diagnose::ROWS = 10;
```

```
bool Diagnose::trace_on = true;
```

```
Diagnose::Diagnose()
```

```
{  
  
}  
}
```

//全部都是static成员变量，所以没有什么需要在构造函数中初始化的。

```
Diagnose::~~Diagnose()
```

```
{  
  
}  
}
```

//this is an empty dtor



```
void Diagnose::TraceOn()
{ // 开启日志显示区
    Diagnose::trace_on = 1;
}
```

```
void Diagnose::TraceOff()
{ // 关闭日志显示区
    Diagnose::trace_on = 0;
}
```

```
void Diagnose::WriteChar(const char ch) // 在光标的位置上写字符ch
{
    if(Diagnose::m_Column >= Diagnose::COLUMNS)
        NextLine(); // 换行

    if(Diagnose::m_Row >= Diagnose::SCREEN_ROWS)
        Diagnose::ClearScreen(); // 清屏

    Diagnose::m_VideoMemory[Diagnose::m_Row * COLUMNS + Diagnose::m_Column] =
        (unsigned char) ch | Diagnose::COLOR; // 写显存
    Diagnose::m_Column++; // 光标右移
}
```



```
void Diagnose::NextLine()
{
    m_Row += 1;
    m_Column = 0;
}
```

```
void Diagnose::ClearScreen()
{
    unsigned int i;

    Diagnose::m_Row = Diagnose::SCREEN_ROWS - Diagnose::ROWS;
    Diagnose::m_Column = 0;

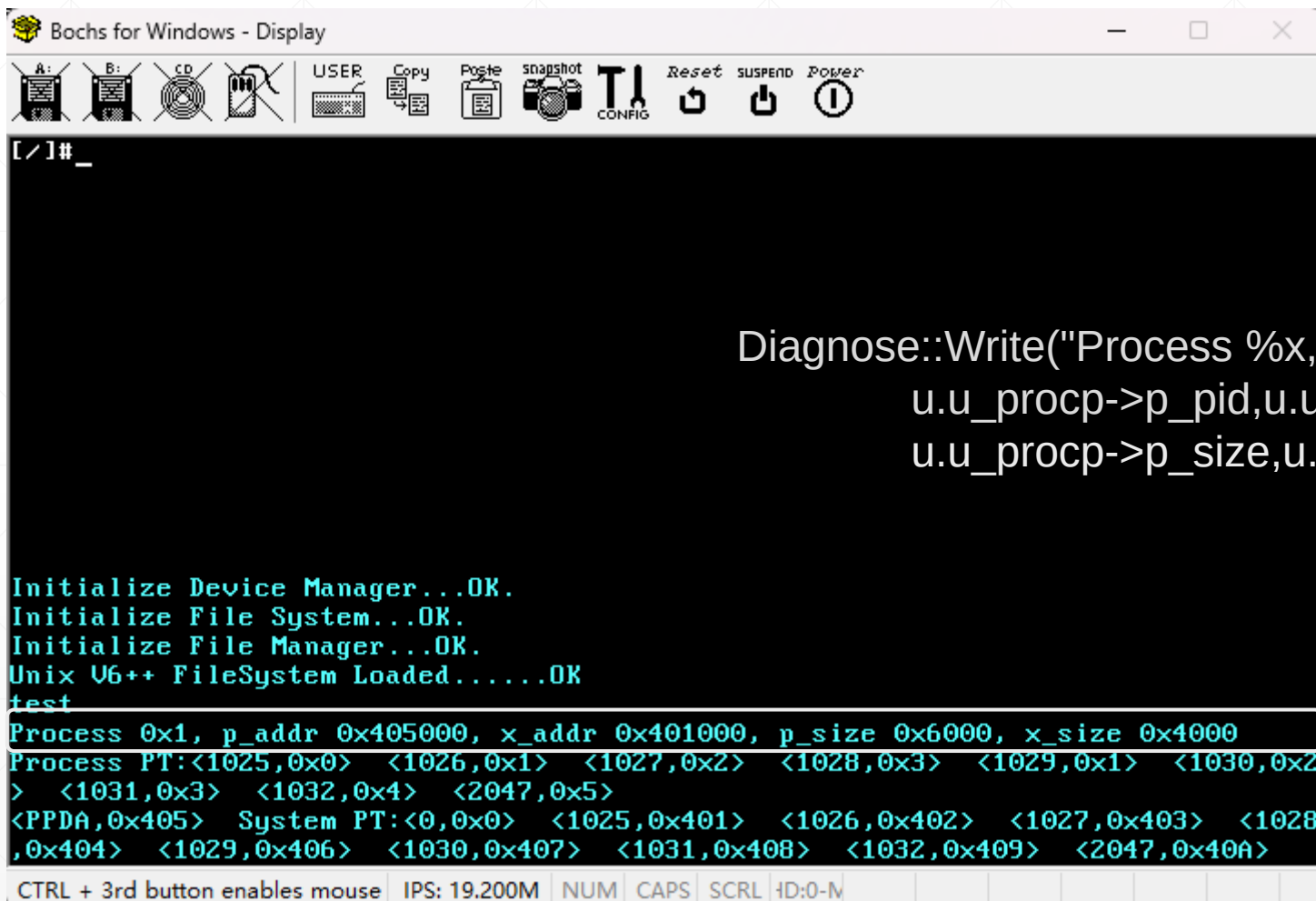
    for(i = 0; i < (COLUMNS * ROWS); i++)
    {
        Diagnose::m_VideoMemory[i + m_Row * COLUMNS] = (unsigned char) ' ' | Diagnose::COLOR;
    }
}
```



```
void Diagnose::PrintInt(unsigned int value, int base) // value, 整数数值。 base, 10表示10进制; 16表示16进制
{ // 这个算法, 好美、好美!!!
    static char Digits[] = "0123456789ABCDEF";
    int i;

    if((i = value / base) != 0)
        PrintInt(i, base);
    WriteChar(Digits[value % base]);
}
```

Diagnose::Write(.....), 内核使用的格式化输出函数, 和printf类似



```
[/]#_

Diagnose::Write("Process %x, p_addr %x, x_addr %x, p_size %x, x_size %x\n",
                u.u_procp->p_pid,u.u_procp->p_addr,u.u_procp->p_textp->x_caddr,
                u.u_procp->p_size,u.u_procp->p_textp->x_size);

Initialize Device Manager...OK.
Initialize File System...OK.
Initialize File Manager...OK.
Unix V6++ FileSystem Loaded.....OK
test
Process 0x1, p_addr 0x405000, x_addr 0x401000, p_size 0x6000, x_size 0x4000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x3> <1029,0x1> <1030,0x2>
> <1031,0x3> <1032,0x4> <2047,0x5>
<PPDA,0x405> System PT:<0,0x0> <1025,0x401> <1026,0x402> <1027,0x403> <1028,0x404>
<1029,0x406> <1030,0x407> <1031,0x408> <1032,0x409> <2047,0x40A>

CTRL + 3rd button enables mouse  IPS: 19.200M NUM CAPS SCRL ID:0-N
```

格式化输出的字符串

Diagnose::Write(.....), 实现

```
void Diagnose::Write(const char* fmt, ...)
{
    if ( false == Diagnose::trace_on )
        return;

    unsigned int * va_arg = (unsigned int *)&fmt + 1;
    const char * ch = fmt;

    while(1)
    {
        while(*ch != '%' && *ch != '\n')
        {
            if(*ch == '\0')
                return;
            if(*ch == '\n')
                break;
            .....
        }
    }
}
```

能够输出格式化后的字符串，目前只能识别一些
%d %x %s 和%n; 没有检查错误功能，% 和 值匹配要自己注意

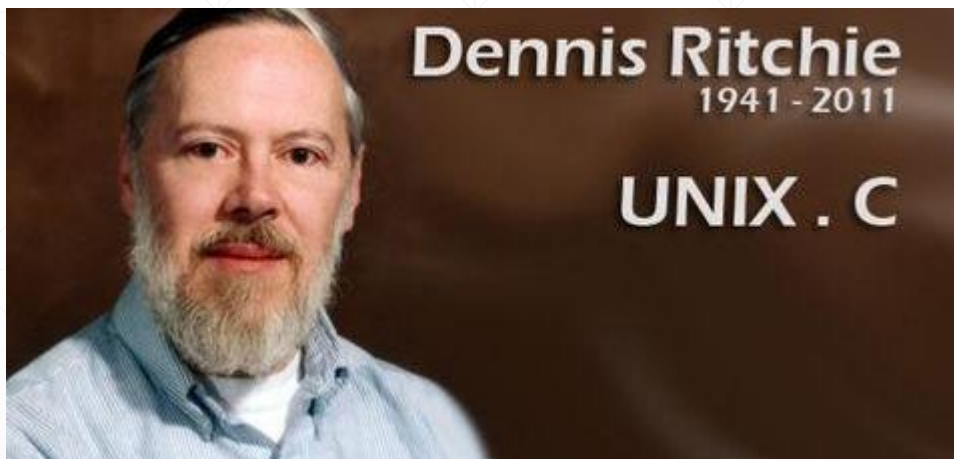
print这个函数很有趣，它的实参，数量不定。那么printf栈帧的实参区就不好处理了是吗。这个函数好神奇，编译器是怎么处理的 ??? 这是里奇的设计，大师是怎么做的。你有没有更好的实现。附加题作业。



使va_arg中存放参数fmt的“后一个参数”所在的内存地址。fmt的内容本身是字符串的首地址(这不是我们想要的)，而&fmt + 1则是下一个参数的地址。

参考UNIX v6中的函数
prf.c/printf(fmt,x1,x2,x3,x4,x5,x6,x7,x8,x9,xa,xb,xc)

C语言 和 Unix之父



```
#include <stdio.h>

void main() {
    printf("Hello, World/n");
}
```

附录

ASCII码表完整版



ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符
0	NUL	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	.	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	X	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL